

FastAPI & Scikit-learn

Technical Integration Guide

Generated: June 08, 2026

Table of Contents

1. Introduction
2. FastAPI Fundamentals
3. Scikit-learn Fundamentals
4. Integration Architecture
5. Building ML APIs
6. Best Practices & Deployment

1. Introduction

FastAPI is a modern, high-performance web framework for building APIs with Python 3.7+, while scikit-learn is a powerful machine learning library for data mining and analysis. This guide explores how to integrate these two frameworks to build production-ready machine learning APIs.

2. FastAPI Fundamentals

2.1 Key Features

- Fast: High performance comparable to Node.js and Go
- Easy to use: Intuitive and easy to learn
- Standards-based: OpenAPI and JSON Schema support
- Async support: Built-in async/await capabilities
- Automatic documentation: Interactive API docs (Swagger UI, ReDoc)
- Type hints: Full support for Python type hints

2.2 Basic Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    description: str = None

@app.get("/")
async def read_root():
    return {"message": "Hello, World!"}

@app.post("/items/")
async def create_item(item: Item):
    return item
```

3. Scikit-learn Fundamentals

3.1 Core Components

- Estimators: Objects that learn from data (fit method)
- Transformers: Objects that transform data (transform method)
- Predictors: Objects that make predictions (predict method)
- Pipelines: Combine multiple steps into a single workflow
- Model Selection: Tools for cross-validation and hyperparameter tuning

3.2 Model Training Example

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Create pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(n_estimators=100))
])

# Train model
pipeline.fit(X_train, y_train)

# Make predictions
predictions = pipeline.predict(X_test)
accuracy = pipeline.score(X_test, y_test)
```

4. Integration Architecture

4.1 Design Pattern

The typical architecture involves: (1) Model training and persistence using scikit-learn, (2) API endpoints exposed via FastAPI, (3) Request validation using Pydantic models, (4) Async request handling for better concurrency.

4.2 Complete ML API Example

```
from fastapi import FastAPI
from pydantic import BaseModel
import pickle
from sklearn.ensemble import RandomForestClassifier

app = FastAPI()

# Load pre-trained model
with open('model.pkl', 'rb') as f:
    model = pickle.load(f)

class PredictionInput(BaseModel):
    feature1: float
    feature2: float
    feature3: float
    feature4: float

class PredictionOutput(BaseModel):
    prediction: int
    probability: float

@app.post("/predict", response_model=PredictionOutput)
async def predict(input_data: PredictionInput):
    features = [
        input_data.feature1, input_data.feature2,
        input_data.feature3, input_data.feature4
    ]

    prediction = model.predict(features)[0]
    probability = model.predict_proba(features)[0].max()

    return PredictionOutput(
        prediction=int(prediction),
        probability=float(probability)
    )

@app.get("/health")
async def health_check():
    return {"status": "healthy"}
```

5. Best Practices & Deployment

5.1 Model Management

- Version control: Track model versions with metadata
- Model persistence: Use joblib or pickle for serialization
- Input validation: Validate input data types and ranges
- Error handling: Implement proper exception handling
- Logging: Log predictions for monitoring and debugging
- Caching: Cache model predictions for repeated inputs

5.2 Performance Optimization

```
# Load model once at startup
from fastapi import FastAPI
from contextlib import asynccontextmanager

model = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load model on startup
    global model
    model = joblib.load('model.pkl')
    yield
    # Cleanup on shutdown

app = FastAPI(lifespan=lifespan)
```

5.3 Deployment Considerations

- Use Uvicorn or Gunicorn as ASGI servers
- Containerize with Docker for consistency
- Implement request rate limiting
- Add authentication (JWT, OAuth2)
- Monitor API performance and model accuracy
- Plan for model updates and versioning

6. Advanced Topics

6.1 Async Predictions with Batch Processing

```
from fastapi import BackgroundTasks
import asyncio

@app.post("/predict-batch")
async def predict_batch(inputs: List[PredictionInput],
                        background_tasks: BackgroundTasks):
    task_id = generate_task_id()
    background_tasks.add_task(
        process_batch,
        inputs,
        task_id
    )
    return {"task_id": task_id}

@app.get("/result/{task_id}")
async def get_result(task_id: str):
    return get_cached_result(task_id)
```

6.2 Multi-Model Ensemble

```
from sklearn.ensemble import VotingClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

ensemble = VotingClassifier(
    estimators=[
        ('svc', SVC(kernel='rbf')),
        ('dt', DecisionTreeClassifier()),
        ('knn', KNeighborsClassifier(n_neighbors=5))
    ],
    voting='soft'
)

ensemble.fit(X_train, y_train)
```

References & Resources

- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- Scikit-learn Documentation: <https://scikit-learn.org/>
- Pydantic Models: <https://docs.pydantic.dev/>
- Uvicorn ASGI Server: <https://www.uvicorn.org/>